Aula 11 — Análise Descendente Recursiva

Compiladores I

FACOM - Faculdade de Computação

Bloco 4: Análise Sintática

Duração estimada: 70–80 minutos

Revisão da Aula 10

- FIRST e FOLLOW $\rightarrow$ tabela de análise preditiva;
- Gramáticas LL(1).

Pergunta rápida

Por que precisamos eliminar recursão à esquerda antes de implementar um parser LL(1)?

Objetivos de aprendizagem

- Implementar um analisador sintático por descida recursiva a partir de uma gramática LL(1);
- Mapear produções gramaticais em procedimentos de análise;
- Incorporar tratamento básico de erros sintáticos.

Sumário da aula

Descida recursiva

A ideia central

Um procedimento por não-terminal

Para cada não-terminal A da gramática, criamos um procedimento `A()` que: (1) olha o token atual (lookahead); (2) decide qual produção de A aplicar (usando FIRST); (3) chama os procedimentos correspondentes aos símbolos do lado direito da produção escolhida.

É a implementação *manual* mais direta de um parser LL(1) — exatamente a abordagem usada no Micro C (Cap. 3.5).

Exemplo: parser recursivo para expressões (Micro C)

Exemplo — Gramática já sem recursão à esquerda

```
expr   -> termo expr'
expr'  -> '+' termo expr' | '-' termo expr' | eps
termo  -> fator termo'
termo'-> '*' fator termo' | '/' fator termo' |
    eps
fator  -> NUM | ID | '(' expr ')'
```

```
No* expr(void) {
    No* esq = termo();
    return expr_linha(esq);
}
No* expr_linha(No* esq) {
    if (token_atual == TK_PLUS || token_atual == TK_MINUS) {
        TokenType op = token_atual; avancar();
        No* dir = termo();
        return expr_linha(novo_no_binario(op, esq, dir));
    }
    return esq; /* caso eps */
}
```

Mapeamento produção → procedimento

- Símbolo **terminal** na produção ⇒ chamada a `esperar(TOKEN)`, que verifica e consome o token;
- Símbolo **não-terminal** ⇒ chamada recursiva ao procedimento daquele não-terminal;
- Alternativas (`|`) ⇒ uma estrutura `if/else` (ou `switch`) decidida pelo FIRST de cada alternativa.

Pergunta rápida

No procedimento `expr_linha` do slide anterior, qual parte do código corresponde à produção `expr' -> eps`?

Tratamento de erros no parser recursivo

- A função `esperar(TOKEN)` verifica se o token atual é o esperado; se não for, reporta erro sintático (token esperado x encontrado, posição);
- Estratégia comum: ao detectar erro, avançar tokens até um ponto de sincronização (ex: próximo ;) e continuar, para detectar múltiplos erros em uma única passada;
- No Micro C, essa lógica está distribuída entre as funções de processamento de tokens esperados (Cap. 3.8.1).

Desafio em sala

Desafio em sala

Escreva (em pseudocódigo ou C) o procedimento `fator()` para a gramática do slide 5, que deve: reconhecer `NUM`, `ID`, ou `'('` `expr` `')'`, retornando o nó da ASA correspondente e reportando erro se nenhuma das três alternativas correspondentes ao token atual.

Síntese da aula

- A descida recursiva mapeia diretamente cada não-terminal em um procedimento;
- Terminais viram chamadas a `esperar()`; não-terminais viram chamadas recursivas;
- O tratamento de erros usa sincronização para continuar a análise após um erro.

Próxima aula: a outra família de analisadores — deslocamento-redução.

Teste rápido — Aula 11

Questões teóricas

1. Por que cada não-terminal da gramática vira um procedimento no parser recursivo descendente?
2. O que a função `esperar(TOKEN)` deve fazer quando o token atual não é o esperado?

Questão prática

3. Escreva o procedimento `termo()` correspondente às produções `termo -> fator termo'` e `termo' -> '*' fator termo' | '/' fator termo' | eps`.